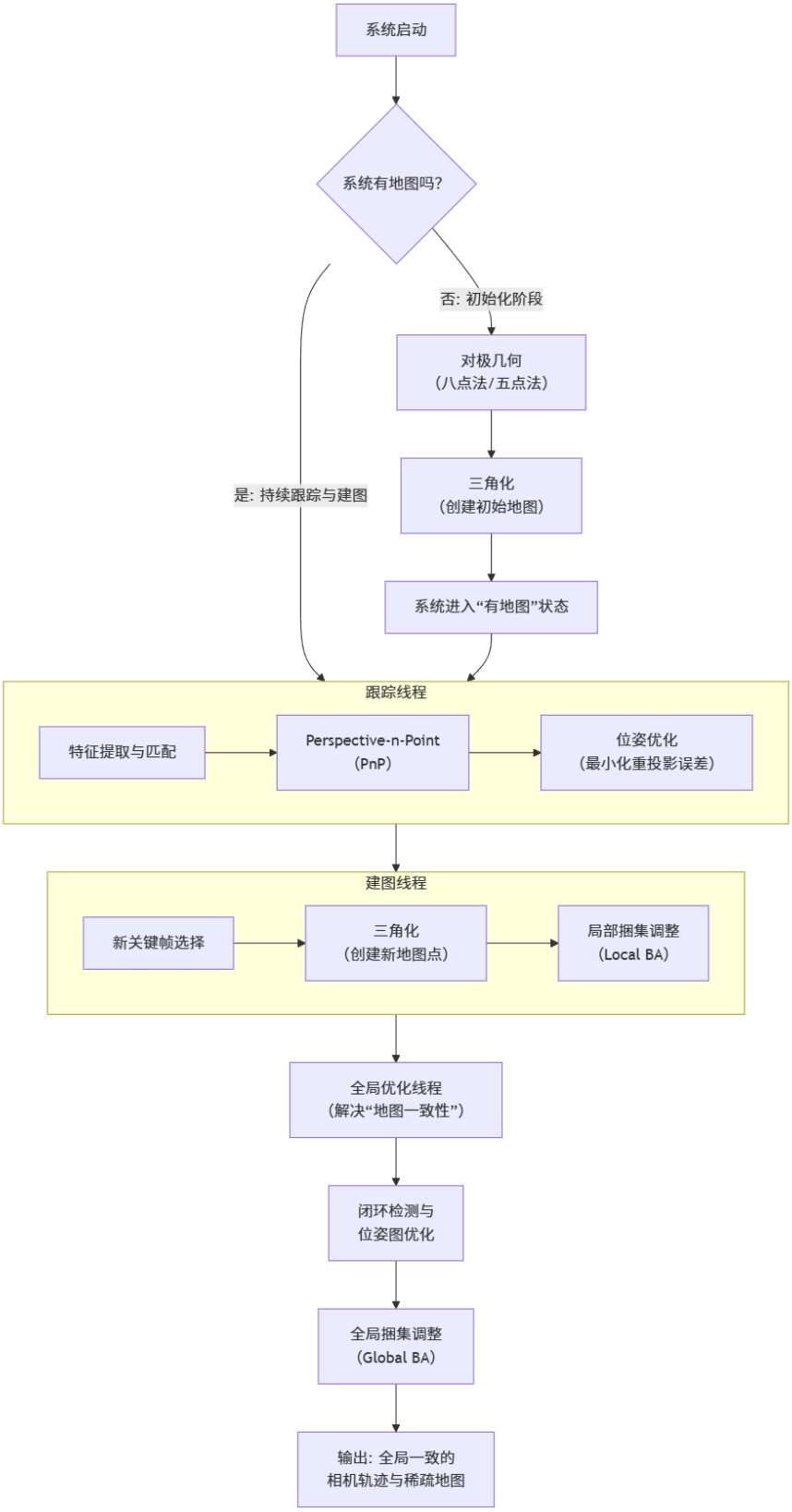
# 0.Perface

在上一篇文章中，我们深入学习了基于滤波方法的 MonoSLAM，理解了 EKF 如何通过预测-更新循环来估计相机状态和地图点位置。然而，滤波方法存在一个根本性的缺陷：误差会随着运动不断累积，且无法被修正。当相机走过一个长距离后，整个轨迹和地图会发生严重的漂移。

本文将介绍现代视觉 SLAM 的核心框架——基于关键帧和优化的方法，以 ORB-SLAM 为典型代表。这种方法通过“闭环检测-位姿图优化-全局 BA”的后端优化流程，主动检测并修正累计误差，从而构建全局一致的大尺度地图。


系统启动
系统有地图吗?
否: 初始化阶段
对极几何
(八点法/五点法)
三角化
(创建初始地图)
系统进入“有地图”状态
是: 持续跟踪与建图
跟踪线程
特征提取与匹配
Perspective-n-Point
(PnP)
位姿优化
(最小化重投影误差)
建图线程
新关键帧选择
三角化
(创建新地图点)
局部捆集调整
(Local BA)
全局优化线程
(解决“地图一致性”)
闭环检测与
位姿图优化
全局捆集调整
(Global BA)
输出: 全局一致的
相机轨迹与稀疏地图

# 1.Foundation：从滤波到优化

## 1.1. 滤波方法的局限性

在 MonoSLAM 的 EKF 框架中，状态估计是递推进行的：每一时刻的状态估计都依赖于上一时刻的结果。这种递推结构带来了几个关键问题：

计算复杂度：状态向量的维度为$13+3N$，其中$N$是地图点数量。当$N$增大时，协方差矩阵的更新需要$O(N^2)$的计算量。更严重的是，每次观测更新都会影响整个状态向量，导致计算负担随地图规模急剧增长。

信息丢失：EKF 在每个时刻只维护当前状态的估计和协方差，历史观测信息被“压缩”进了当前状态。这意味着一旦某个估计出现偏差，它的影响会永久保留，无法被后续观测修正。

线性化误差：EKF 通过一阶泰勒展开来线性化非线性系统。当系统非线性程度较高时，线性化误差会累积，导致估计偏离真实值。

## 1.2. 优化方法的核心思想

优化方法的出发点是：与其递推地维护一个不断增长的状态，不如定期“回顾”所有历史信息，进行全局优化。

关键帧机制：不是每一帧都参与优化，而是选择具有代表性的关键帧。关键帧之间有足够的视差，能够提供稳定的几何约束。这大大减少了需要优化的变量数量。

延迟优化：不是实时更新所有状态，而是在特定时机（如闭环检测后）进行批量优化。这使得我们可以使用更复杂的优化算法，如非线性最小二乘。

全局一致性：优化时同时考虑所有历史观测，能够检测并修正累积误差，保证地图的全局一致性。

# 2.Bundle Adjustment：捆集调整

## 2.1. BA 的数学形式

捆集调整（Bundle Adjustment）是视觉 SLAM 中最核心的优化技术。它的目标是最小化所有观测的重投影误差。

目标函数：

设我们有$M$个关键帧和$N$个地图点。对于第$i$个关键帧观测到的第$j$个地图点，设观测像素坐标为$z_{\mathrm{i,j}}=\begin{pmatrix}u_{\mathrm{i,j}}\\ v_{\mathrm{i,j}}\end{pmatrix}$，则 BA 的目标函数为：

$$E=\sum_{i=1}^{M}\sum_{j=1}^{N}\delta_{\mathrm{i,j}}\parallel z_{\mathrm{i,j}}-\pi\left(T_i,P_j\right)\parallel^2$$

其中：

- $T_i=[R_i,t_i]$是第$i$个关键帧的位姿（6 自由度）
- $P_j=\begin{pmatrix}X_j\\ Y_j\\ Z_j\end{pmatrix}$是第$j$个地图点的世界坐标（3 自由度）
- $\pi\left(T_i,P_j\right)$是将地图点$P_j$投影到关键帧$i$的像素坐标
- $\delta_{\mathrm{i,j}}$是指示函数，若地图点$j$在关键帧$i$中可见则为 1，否则为 0

投影函数$\pi$的展开：

$$\pi(T_i, P_j) = K(R_i P_j + t_i)$$

更具体地，设$P_c = R_i P_j + t_i = \begin{pmatrix} X_c \\ Y_c \\ Z_c \end{pmatrix}$，则：

$$u = f_x(X_c/Z_c) + c_x$$

$$v = f_y(Y_c/Z_c) + c_y$$

优化变量：

BA 同时优化两类变量：
- 关键帧位姿：$T_1, T_2, ..., T_M$，共$6M$个参数
- 地图点坐标：$P_1, P_2, ..., P_N$，共$3N$个参数

总共有$6M + 3N$个待优化参数。

## 2.2. BA 的求解：非线性最小二乘

BA 本质上是一个非线性最小二乘问题，通常使用高斯-牛顿法或 Levenberg-Marquardt 算法求解。

线性化：

设当前估计为$x = [T_1, ..., T_M, P_1, ..., P_N]$，我们需要找到增量$\Delta x$使得目标函数最小化。将目标函数在当前点线性化：

$$E(x + \Delta x) \approx E(x) + J\Delta x$$

其中$J$是雅可比矩阵，其元素为：

$$J_{\mathrm{i,j,k}} = (\delta e_{\mathrm{i,j}})/(\delta x_k)$$

这里$e_{\mathrm{i,j}} = z_{\mathrm{i,j}} - \pi(T_i, P_j)$是重投影误差。

正规方程：

最小化线性化后的目标函数，得到正规方程：

$$J^T J\Delta x = -J^T e$$

求解这个线性方程组得到增量$\Delta x$，然后更新$x = x + \Delta x$，迭代直到收敛。

稀疏性：

BA 问题的雅可比矩阵具有特殊的稀疏结构。每个重投影误差$e_{\mathrm{i,j}}$只依赖于两个变量：关键帧位姿$T_i$和地图点坐标$P_j$。因此，雅可比矩阵的大部分元素为零。

这种稀疏性使得我们可以使用 Schur 消元技巧，大幅降低计算复杂度。具体来说，我们可以先消去地图点变量，只求解关键帧位姿的增量，然后再回代求解地图点的增量。

## 2.3. BA 的局部最小值问题

虽然 BA 是一个强大的优化工具，但它存在一个关键问题：容易陷入局部最小值。

问题根源：

BA 的目标函数是非凸的。当初始值偏离真实值较远时，优化算法可能收敛到一个局部最小值，而不是全局最小值。

在 **SLAM** 中的表现：

当相机运动距离较长，累积漂移较大时，直接进行全局 BA 往往无法得到正确结果。这是因为闭环约束与局部约束之间存在矛盾，而 BA 可能陷入一个满足局部约束但违反全局约束的局部最优解。

解决方案：

这就是为什么现代 SLAM 采用“先位姿图优化，后 BA”的策略。位姿图优化能够纠正宏观的轨迹漂移，为 BA 提供一个良好的初始值，从而避免陷入局部最小值。

# 3.Pose Graph Optimization：位姿图优化

## 3.1. 位姿图的构建

位姿图是一种简化的图模型，其中节点是关键帧的位姿，边是位姿之间的相对变换约束。

节点：每个关键帧$i$对应一个节点，其状态为位姿$T_i = [R_i, t_i]$。

边：两个关键帧$i$和$j$之间的边表示它们的相对位姿约束。这条边可以来自：
- 时序约束：相邻关键帧之间通过特征匹配计算的相对运动
- 闭环约束：检测到回环时，当前帧与历史关键帧之间的相对位姿

边的数学表示：

设关键帧$i$和$j$之间的相对位姿为$T_{\mathrm{i,j}}$，则约束方程为：

$$T_{\mathrm{i,j}} = T_j T_i^{-1}$$

其中位姿复合操作表示为矩阵乘法。更具体地，设$T_i = [R_i, t_i]$，$T_j = \left[R_j, t_j\right]$，则：

$$R_{\mathrm{i,j}} = R_j R_i^T$$

$$t_{\mathrm{i,j}} = t_j - R_j R_i^T t_i$$

## 3.2. 位姿图优化的目标函数

位姿图优化的目标是最小化所有边约束的误差。

目标函数：

$$E = \sum_{\mathrm{i,j}} \| \ e_{\mathrm{i,j}} \ \|^2_{\{\Sigma_{\mathrm{i,j}}\}}$$

其中$e_{\mathrm{i,j}}$是边$(i, j)$的误差向量，$\Sigma_{\mathrm{i,j}}$是约束的不确定性协方差矩阵。

误差向量的定义：

对于位姿误差，我们需要定义一个合适的误差度量。常用的方法是使用李代数表示：

$$e_{\mathrm{i,j}} = \log\left(T_{\mathrm{i,j}} \times \left(T_j T_i^{-1}\right)^{-1}\right)$$

这里$\log$是将$\mathrm{SE}(3)$矩阵映射到$\mathrm{se}(3)$李代数的对数映射，结果是一个 6 维向量（3 维旋转误差 + 3 维平移误差）。

权重矩阵：

$\| e \|_{\Sigma}^2 = e^T \Sigma^{-1} e$是马氏距离，它考虑了约束的不确定性。如果某个约束的协方差$\Sigma$较大，说明这个约束不可靠，在优化中权重较小。

### 3.3. 位姿图优化的求解

位姿图优化同样是非线性最小二乘问题，可以使用高斯-牛顿法或 g2o 等图优化库求解。

雅可比矩阵的计算：

误差$e_{\mathrm{i,j}}$对位姿$T_i$和$T_j$的雅可比矩阵可以通过李代数的链式法则计算：

$$\left(\delta e_{\mathrm{i,j}}\right)/(\delta \xi_i) = -J_{\mathrm{adj}} J_{\mathrm{r}}$$

$$\left(\delta e_{\mathrm{i,j}}\right)/\left(\delta \xi_j\right) = J_{\mathrm{r}}$$

其中$\xi_i$和$\xi_j$分别是$T_i$和$T_j$的李代数表示，$J_{\mathrm{adj}}$是伴随矩阵的雅可比，$J_{\mathrm{r}}$是右乘雅可比。

优化效率：

位姿图优化的变量只有关键帧位姿，数量远小于 BA 中的变量数（BA 还包括地图点）。因此，位姿图优化的计算效率远高于 BA，适合在闭环检测后快速纠正全局轨迹。

### 3.4. 单目 SLAM 中的 7 自由度优化

在单目 SLAM 中，位姿图优化需要考虑尺度问题。

尺度模糊性：

单目 SLAM 无法确定绝对尺度。整个轨迹和地图可以乘以一个任意尺度因子$s$，而所有观测仍然一致。

7 自由度优化：

为了纠正尺度漂移，位姿图优化需要增加一个尺度自由度。每个关键帧的位姿变为$T_i = [s_i R_i, t_i]$，或者更简单地，假设全局尺度$s$，优化变量为$[s, R_1, t_1, R_2, t_2, ...]$。

**“Sim(3)”**群：

数学上，带尺度的位姿变换属于$\mathrm{Sim}(3)$群（相似变换群），其元素可以表示为：

$$S = \begin{pmatrix} sR & t \\ 0 & 0 \quad 0 \quad 1 \end{pmatrix}$$

其中$s$是尺度因子，$R \in \mathrm{SO}(3)$是旋转矩阵，$t$是平移向量。

$\mathrm{Sim}(3)$的李代数$\mathrm{sim}(3)$是 7 维的，对应 7 个自由度：3 个旋转、3 个平移、1 个尺度。

## 4.Loop Closing：闭环检测

### 4.1. 闭环检测的意义

闭环检测是现代 SLAM 区别于早期方法的关键模块。它的作用是识别相机是否回到了之前访问过的位置。

为什么需要闭环检测：

在长距离运动中，累积误差会导致轨迹漂移。如果相机回到起点，我们期望轨迹应该闭合。然而，由于漂移，起点和终点的估计位姿可能相差很远。闭环检测提供了“我回来过这里”的信息，使得系统能够检测并修正累积误差。

闭环检测的挑战：

1. 识别问题：如何判断当前场景与历史场景是否相同？
2. 效率问题：如何快速在大量历史关键帧中检索？
3. 误检问题：如何避免将相似但不同的场景误判为闭环？

## 4.2. 词袋模型（Bag of Words）

词袋模型是闭环检测的主流方法，ORB-SLAM 等系统都采用了这一技术。

视觉词典的构建：

1. 从大量图像中提取特征点（如 ORB 特征）
2. 对特征描述子进行聚类（如 K-means），得到$K$个聚类中心，称为“视觉单词”
3. 每个视觉单词代表一类局部图像特征
4. 可以构建多级词典（如树状结构），提高检索效率

图像的词袋表示：

给定一幅图像，提取其所有特征点，将每个特征点分配到最近的视觉单词。图像可以表示为一个稀疏向量：

$$v = [n_1, n_2, ..., n_K]$$

其中$n_i$是该图像中属于第$i$个视觉单词的特征点数量。

相似度计算：

两幅图像的相似度可以通过它们词袋向量的距离来度量。常用的度量包括：
- L1 距离
- L2 距离
- TF-IDF 加权的距离

TF-IDF（Term Frequency-Inverse Document Frequency）考虑了视觉单词的重要性：出现频率高的单词（如边缘、角点）权重低，出现频率低的单词（如独特纹理）权重高。

## 4.3. 闭环检测流程

ORB-SLAM 的闭环检测流程如下：

步骤 1：关键帧数据库检索

当新关键帧产生时，计算其词袋向量，在关键帧数据库中检索相似的关键帧。

步骤 2：候选筛选

检索结果中可能存在误检，需要进一步筛选：
- 连续性检验：要求连续多个关键帧都检索到同一个候选闭环帧
- 几何验证：通过特征匹配和基础矩阵检验，确认候选帧与当前帧确实观测到相同场景

步骤 3：计算相对位姿

确认闭环后，通过特征匹配计算当前帧与闭环帧之间的相对位姿$T_{\text{loop}}$。这个相对位姿就是闭环约束。

**步骤 4：位姿图优化**

将闭环约束添加到位姿图中，执行位姿图优化，纠正累积漂移。

**步骤 5：全局 BA**

在位姿图优化后，执行全局 BA，精细优化所有关键帧位姿和地图点坐标。

# 5.Scale Problem：尺度问题

## 5.1. 单目 SLAM 的尺度模糊性

单目相机是一个 2D 传感器，它拍摄的图像丢失了深度信息。这导致了单目 SLAM 固有的尺度模糊性。

投影方程分析：

设相机内参为$K$，世界点$P = \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}$投影到像素$\begin{pmatrix} u \\ v \end{pmatrix}$：

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix}^T \propto K \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}^T$$

如果我们将世界点坐标乘以尺度因子$s$，即$P' = sP$，同时将相机平移也乘以$s$，即$t' = st$，则投影结果不变：

$$K(R(sP) + st) = sK(RP + t) \propto K(RP + t)$$

这意味着，对于单目 SLAM，我们无法区分真实场景和按比例缩放的场景。

初始化时的尺度设定：

在 SLAM 初始化时，系统需要设定一个初始尺度。通常的做法是将第一个地图点的深度设为某个固定值（如 1 米）。这个尺度是任意的，没有物理意义。

## 5.2. 尺度漂移

更严重的问题是尺度漂移——在运动过程中，地图的尺度会逐渐变化。

漂移的来源：

1. 深度估计误差：三角化得到的地图点深度有误差
2. 旋转估计误差：旋转误差会影响后续帧的位姿估计
3. 递推累积：误差在递推过程中不断累积

漂移的表现：

假设真实轨迹是一个正方形，每边 10 米。由于尺度漂移，SLAM 估计的轨迹可能变成一个长方形，或者整体放大/缩小。

数学分析：

设真实尺度为$s_{\text{true}}$，估计尺度为$s_{\text{est}}$。尺度漂移可以建模为：

$$s_{\text{est}} = s_{\text{true}}(1 + \varepsilon)$$

其中$\varepsilon$是漂移量。漂移量会随运动距离累积，可以近似为：

$$\varepsilon \propto \sqrt{D}$$

其中$D$是运动距离。这意味着漂移与运动距离的平方根成正比。

### 5.3. 全局 BA 对尺度的修正

全局 BA 能够检测并修正尺度漂移，但前提是有闭环约束。

闭环约束的作用：

当检测到闭环时，系统获得了强约束：当前位姿应该与历史位姿一致。这个约束与局部估计之间存在矛盾，BA 需要调整所有变量来最小化总误差。

7 自由度优化：

在单目 SLAM 的全局优化中，需要优化 7 个自由度：3 个旋转、3 个平移、1 个尺度。这可以通过$\mathrm{Sim}(3)$群的优化来实现。

尺度修正的数学形式：

设优化前的轨迹为$\{T_1, T_2, ..., T_n\}$，优化后的轨迹为$\{T_1', T_2', ..., T_n'\}$。尺度修正可以表示为：

$$T_i' = ST_i$$

其中$S \in \mathrm{Sim}(3)$是相似变换，包含尺度因子$s$。

## 6.Triangulation：三角化建图

### 6.1. 三角化的几何原理

三角化是从多视角观测恢复 3D 点位置的核心技术。

基本原理：

设相机在两个位置$C_1$和$C_2$观测到同一个 3D 点$P$。从每个相机位置出发，可以确定一条射线，两条射线的交点就是$P$的位置。

数学表达：

设相机位姿为$T_1 = [R_1, t_1]$和$T_2 = [R_2, t_2]$，像素观测为$\begin{pmatrix} u_1 \\ v_1 \end{pmatrix}$和$\begin{pmatrix} u_2 \\ v_2 \end{pmatrix}$。

将像素反投影到相机坐标系：

$$x_1 = K^{-1} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}^T$$

$$x_2 = K^{-1} \begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix}^T$$

这两条射线在世界坐标系中为：

$$L_1 : P = t_1 + \lambda_1 R_1^T x_1$$

$$L_2 : P = t_2 + \lambda_2 R_2^T x_2$$

其中$\lambda_1$和$\lambda_2$是深度参数。

## 6.2. 三角化的线性解法

实际中，由于噪声，两条射线不会精确相交。我们需要找到最接近两条射线的点。

线性方法：

将两条射线的方程写成矩阵形式：

$$\begin{pmatrix} R_1^T x_1 & -R_2^T x_2 \end{pmatrix} \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix}^T = t_2 - t_1$$

这是一个超定方程组（3 个方程，2 个未知数），可以用最小二乘法求解：

$$\begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix}^T = (A^T A)^{-1} A^T b$$

其中$A = \begin{pmatrix} R_1^T x_1 & -R_2^T x_2 \end{pmatrix}$，$b = t_2 - t_1$。

深度计算：

求得$\lambda_1$和$\lambda_2$后，可以计算 3D 点坐标：

$$P = t_1 + \lambda_1 R_1^T x_1$$

或者取两个估计的平均：

$$P = (t_1 + \lambda_1 R_1^T x_1 + t_2 + \lambda_2 R_2^T x_2)/2$$

## 6.3. 三角化的质量评估

三角化的质量取决于多个因素：

视差（**Parallax**）：

视差是两条射线之间的夹角。视差越大，深度估计越准确。当视差接近 0 时，深度估计变得非常不稳定。

视差的计算：

$$\theta = \arccos((R_1^T x_1) \cdot (R_2^T x_2))$$

重投影误差：

三角化后，将 3D 点投影回两个视角，计算与原始观测的偏差。重投影误差大说明三角化质量差。

深度比：

如果$\lambda_1$和$\lambda_2$差异很大，说明三角化可能有问题（如匹配错误）。

## 6.4. 尺度为何变化

回到核心问题：为什么三角化会导致地图尺度变化?

误差传播链：

1. 初始时刻，系统设定一个任意尺度$s_0$
2. 相机移动后，三角化新地图点。由于位姿估计有误差，新地图点的深度估计也有误差
3. 当相机用这些“深度略有偏差”的地图点估计新位姿时，会调整估计的移动距离来匹配观测
4. 这个调整会改变系统的内部尺度

数学分析：

设真实深度为$Z$，估计深度为$Z' = Z(1+\delta)$。当相机移动距离$d$后，用这个地图点估计新位姿：

真实情况：视差$\alpha = d/Z$

估计情况：视差$\alpha' = d'/Z' = d'/(Z(1+\delta))$

为了匹配观测到的视差，系统会估计：

$$d' = d(1+\delta)$$

即移动距离被错误地放大或缩小了。这个误差会传播到后续所有估计中。

漂移累积：

每次三角化和位姿估计都会引入微小误差。这些误差累积起来，导致整体尺度的漂移。漂移的方向取决于误差的性质：如果深度估计倾向于偏大，尺度会收缩；如果偏小，尺度会膨胀。

# 7.Reprojection Error：重投影误差

## 7.1. 重投影的定义

重投影是将已估计的 3D 地图点，按照当前估计的相机位姿，重新投影到 2D 图像平面上，得到预测的像素位置。

数学表达：

设地图点$P = \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}$在世界坐标系中，相机位姿为$T = [R, t]$，内参为$K$。

重投影过程：

$$P_c = RP + t$$

（变换到相机坐标系）

$$\begin{pmatrix} u \\ v \end{pmatrix} = \pi(KP_c)$$

（投影到像素坐标）

其中$\pi$是透视投影函数：

$$\pi\left(\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}\begin{pmatrix} X_c \\ Y_c \\ Z_c \end{pmatrix}\right) = \begin{pmatrix} f_x X_c/Z_c + c_x \\ f_y Y_c/Z_c + c_y \end{pmatrix}$$

## 7.2. 重投影误差的定义

重投影误差是预测像素位置与实际观测像素位置之间的差异。

$$e = z_{\text{obs}} - z_{\text{pred}}$$

其中$z_{\text{obs}}$是实际观测的像素坐标，$z_{\text{pred}}$是重投影得到的预测坐标。

几何意义：

重投影误差反映了状态估计（相机位姿和地图点坐标）与观测数据之间的一致性。如果估计完全准确，重投影误差应该为零。

在优化中的作用：

SLAM 系统的优化目标就是最小化所有观测的重投影误差之和：

$$E = \sum_i \| z_{\text{obs,i}} - z_{\text{pred,i}} \|^2$$

### 7.3. 重投影误差的雅可比矩阵

为了优化重投影误差，我们需要计算其对状态变量的雅可比矩阵。

对地图点的偏导数：

$$(\delta u)/(\delta P) = (\delta u)/(\delta P_c)(\delta P_c)/(\delta P)$$

其中：

$$(\delta u)/(\delta P_c) = \begin{pmatrix} f_x/Z_c & 0 & -f_x X_c/Z_c^2 \\ 0 & f_y/Z_c & -f_y Y_c/Z_c^2 \end{pmatrix}$$

$$(\delta P_c)/(\delta P) = R$$

因此：

$$(\delta u)/(\delta P) = \begin{pmatrix} f_x/Z_c & 0 & -f_x X_c/Z_c^2 \\ 0 & f_y/Z_c & -f_y Y_c/Z_c^2 \end{pmatrix} R$$

对相机位姿的偏导数：

对平移$t$的偏导数：

$$(\delta u)/(\delta t) = (\delta u)/(\delta P_c)(\delta P_c)/(\delta t) = \begin{pmatrix} f_x/Z_c & 0 & -f_x X_c/Z_c^2 \\ 0 & f_y/Z_c & -f_y Y_c/Z_c^2 \end{pmatrix}$$

对旋转$R$的偏导数（使用李代数表示）：

$$(\delta u)/(\delta \varphi) = (\delta u)/(\delta P_c)[P_c]_\times$$

其中$[P_c]_\times$是$P_c$的反对称矩阵。

## 8.ORB-SLAM 系统框架

### 8.1. 系统概述

ORB-SLAM 是现代视觉 SLAM 的代表作，它完整实现了“跟踪-建图-闭环”三线程架构。

三个并行线程：

1. 跟踪线程（**Tracking**）：实时估计当前帧的位姿，决定是否生成新关键帧
2. 局部建图线程（**Local Mapping**）：处理新关键帧，进行局部 BA，剔除冗余关键帧
3. 闭环线程（**Loop Closing**）：检测闭环，执行位姿图优化和全局 BA

### 8.2. 跟踪线程

跟踪线程是系统的前端，负责实时处理每一帧图像。

主要步骤：

1. **ORB** 特征提取：从图像中提取 ORB 特征点，计算描述子
2. 初始位姿估计：使用上一帧的位姿或运动模型预测当前帧位姿
3. 特征匹配：将当前帧特征与局部地图点进行匹配
4. 位姿优化：通过最小化重投影误差优化当前帧位姿
5. 关键帧判断：根据条件决定是否生成新关键帧

关键帧生成条件：

- 距离上一个关键帧超过一定帧数
- 局部地图点观测数量低于阈值
- 当前帧跟踪到的地图点数量足够

### 8.3. 局部建图线程

局部建图线程负责维护局部地图的一致性。

主要步骤：

1. 关键帧插入：将新关键帧加入地图，更新共视图
2. 地图点筛选：剔除质量差的地图点（观测次数少、重投影误差大）
3. 新地图点三角化：在相邻关键帧之间三角化新地图点
4. 局部 **BA**：优化当前关键帧及其邻居的位姿和地图点
5. 冗余关键帧剔除：删除被其他关键帧覆盖的冗余帧

共视图（**Covisibility Graph**）：

共视图记录了关键帧之间的共视关系。两个关键帧如果有足够多的共同观测地图点，则它们之间存在一条边。共视图用于确定局部 BA 的优化范围。

### 8.4. 闭环线程

闭环线程负责检测和修正累积误差。

主要步骤：

1. 闭环检测：使用词袋模型检索相似关键帧
2. 几何验证：通过 Sim(3)变换验证闭环假设
3. 闭环融合：融合重复地图点，更新共视图
4. 本质图优化：在本质图上执行位姿图优化
5. 全局 **BA**：优化所有关键帧位姿和地图点

本质图（**Essential Graph**）：

本质图是共视图的稀疏子图，包含生成树、共视边和闭环边。它用于高效的位姿图优化。

## 9.Summary

本文从滤波方法的局限性出发，介绍了现代视觉 SLAM 的核心技术：

**BA（Bundle Adjustment）**：通过最小化重投影误差，同时优化相机位姿和地图点坐标。BA 是视觉 SLAM 最核心的优化技术，但容易陷入局部最小值。

位姿图优化：在关键帧之间建立相对位姿约束，通过图优化纠正全局轨迹漂移。位姿图优化比 BA 更高效，适合作为闭环后的快速修正。

闭环检测：使用词袋模型识别重访场景，为系统提供闭环约束。闭环检测是修正累积误差的关键。

尺度问题：单目 SLAM 存在固有的尺度模糊性和漂移问题。通过 Sim(3)优化可以在闭环时修正尺度漂移。

三角化：从多视角观测恢复 3D 点位置，是建图的核心技术。三角化质量受视差、重投影误差等因素影响。

重投影误差：衡量状态估计与观测一致性的核心指标，是 BA 优化的目标函数。

现代视觉 SLAM 的成功，在于将这些技术有机地整合到一个系统中，实现了实时、鲁棒、全局一致的位姿估计和地图构建。